

Lesson 7 Worksheet: The Challenge of Turning – ANSWER KEY

Part 1: Desired Heading from Coordinate Change

Current Position	Next Position	row_diff	col_diff	What Changed	Desired Heading
(0, 0)	(1, 0)	$1 - 0 = 1$	$0 - 0 = 0$	row +1	2 (S)
(2, 3)	(1, 3)	$1 - 2 = -1$	$3 - 3 = 0$	row -1	0 (N)
(1, 1)	(1, 2)	0	1	col +1	1 (E)
(3, 2)	(3, 1)	0	-1	col -1	3 (W)
(0, 3)	(1, 3)	1	0	row +1	2 (S)
(2, 0)	(2, 1)	0	1	col +1	1 (E)
(3, 1)	(2, 1)	-1	0	row -1	0 (N)
(0, 2)	(0, 1)	0	-1	col -1	3 (W)

Part 2: The `turn_to` While Loop

What does `if heading == 4: heading = 0` do, and why is it necessary?

It resets `heading` back to 0 (North) after it increments past the last valid heading. Without this, after turning right from 3 (West), `heading` would become 4, which is not a valid heading. The wrap makes the heading values form a cycle: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0 \rightarrow 1 \rightarrow \dots$ – matching how the compass actually works.

Part 3: Trace `turn_to` by Hand

Scenario	Current	Desired	heading values during turn_to	Total Turns
1	0 (N)	1 (E)	0 → 1	1
2	1 (E)	1 (E)	already there	0
3	2 (S)	0 (N)	2 → 3 → 0 (wrap)	2
4	3 (W)	2 (S)	3 → 0 (wrap) → 1 → 2	3
5	1 (E)	0 (N)	1 → 2 → 3 → 0 (wrap)	3
6	0 (N)	3 (W)	0 → 1 → 2 → 3	3
7	2 (S)	3 (W)	2 → 3	1
8	3 (W)	1 (E)	3 → 0 (wrap) → 1	2

- Which scenarios trigger the wrap from 4 back to 0? **3, 4, 5, 8** (any time the loop crosses the 3 → 0 boundary)
- Maximum number of turns the loop can ever run? **3** (if it needed a 4th turn, the heading would already equal desired)

Part 4: Trace a Path with Heading Updates

Path: [(1, 0), (2, 0), (2, 1), (2, 2), (2, 3)], Starting position: (0, 0), Starting heading: 0 (N)

Step	Position	Next	Desired	Current	turn_to heading values	New Heading
1	(0, 0)	(1, 0)	2 (S)	0 (N)	0 → 1 → 2	2 (S)
2	(1, 0)	(2, 0)	2 (S)	2 (S)	already there	2 (S)
3	(2, 0)	(2, 1)	1 (E)	2 (S)	2 → 3 → 0 (wrap) → 1	1 (E)
4	(2, 1)	(2, 2)	1 (E)	1 (E)	already there	1 (E)
5	(2, 2)	(2, 3)	1 (E)	1 (E)	already there	1 (E)

- How many steps required turning? **2** (steps 1 and 3)
- How many steps ran the while loop zero times? **3** (steps 2, 4, 5)
- At which step did the robot change from row to column movement? **Step 3**

Part 5: Trace a Second Path

Path: [(1, 3), (0, 3), (0, 2), (0, 1), (0, 0)], Starting position: (2, 3), Starting heading: 2 (S)

Step	Position	Next	Desired	Current	turn_to heading values	New Heading
1	(2, 3)	(1, 3)	0 (N)	2 (S)	2 → 3 → 0 (wrap)	0 (N)
2	(1, 3)	(0, 3)	0 (N)	0 (N)	already there	0 (N)
3	(0, 3)	(0, 2)	3 (W)	0 (N)	0 → 1 → 2 → 3	3 (W)
4	(0, 2)	(0, 1)	3 (W)	3 (W)	already there	3 (W)
5	(0, 1)	(0, 0)	3 (W)	3 (W)	already there	3 (W)

Final heading after completing path: **3 (W)**

Part 6: Trace a Third Path

Path: [(1, 2), (1, 3), (2, 3), (3, 3)], Starting position: (1, 1), Starting heading: 0 (N)

Step	Position	Next	Desired	Current	turn_to heading values	New Heading
1	(1, 1)	(1, 2)	1 (E)	0 (N)	0 → 1	1 (E)
2	(1, 2)	(1, 3)	1 (E)	1 (E)	already there	1 (E)
3	(1, 3)	(2, 3)	2 (S)	1 (E)	1 → 2	2 (S)
4	(2, 3)	(3, 3)	2 (S)	2 (S)	already there	2 (S)

- Final heading: **2 (S)**
- Does the first step require turning? **Yes** — The robot starts facing 0 (N), desired is 1 (E). The loop runs once: heading goes 0 → 1.

Part 7: Robot Orientation Diagrams

- Scenario A: Facing 0 (N), desired 1 (E) → loop runs **1** time (0 → 1)
- Scenario B: Facing 1 (E), desired 2 (S) → loop runs **1** time (1 → 2)
- Scenario C: Facing 3 (W), desired 1 (E) → loop runs **2** times (3 → 0 wrap → 1). Wrap triggered? **Yes**
- Scenario D: Facing 2 (S), desired 2 (S) → loop runs **0** times (already there)

Part 8: Heading Logic in Code

```
HEADING_NAMES = ["N", "E", "S", "W"]
```

```
def desired_heading(current_pos, next_pos):
    row_diff = next_pos[0] - current_pos[0]
    col_diff = next_pos[1] - current_pos[1]

    if row_diff == -1:
        return 0      # North
    elif col_diff == 1:
        return 1      # East
    elif row_diff == 1:
        return 2      # South
    elif col_diff == -1:
        return 3      # West
```

Test your understanding:

1. `desired_heading((0,0), (1,0))` returns **2** — `row_diff = 1`, so heading is 2 (South)
2. `desired_heading((2,3), (2,2))` returns **3** — `col_diff = -1`, so heading is 3 (West)
3. `desired_heading((1,1), (0,1))` returns **0** — `row_diff = -1`, so heading is 0 (North)

Reflection

Why is adding 1 and wrapping 4 → 0 enough to handle every turning case?

Because the four headings form a circle. Each right turn moves one position clockwise around that circle, so repeatedly adding 1 eventually reaches any target heading — in at most 3 steps. The wrap from 4 back to 0 closes the circle so “West + one right turn” ends up at North, just like the real compass. The while loop stops automatically when the heading matches the desired value, so the same code handles all 16 combinations (0, 1, 2, or 3 turns) without any special cases.